

CAPSTONE PROJECT

Agentic AI System for Automated Discharge Summaries for Hospitals

Frameworks	LangGraph · Google ADK · Agno
MCP Primitives	Tools · Resources · Prompts · Sampling · Elicitation · Roots
A2A Protocol	Streaming + Non-Streaming + Push Notifications
Observability	LangFuse · RAI Guardrails · HITL Dashboard

1. Project Overview

1.1 Project Description

This project delivers an end-to-end Agentic AI solution for optimizing hospital discharge management by analyzing clinical documents, validating them against hospital Electronic Health Record (EHR) systems and clinical guidelines, and identifying potential care gaps or inconsistencies. It uses an Agentic RAG framework with Human-in-the-Loop (HITL) feedback to enable accurate, context-aware decision support and generates standardized discharge summaries to improve care continuity and reduce errors.

The system coordinates multiple AI agents across three frameworks (LangGraph, Google ADK, Agno), communicates via the Agent-to-Agent (A2A) Protocol (streaming and non-streaming), exposes tools through two MCP servers demonstrating all six MCP primitives (Tools, Resources, Prompts, Sampling, Elicitation, Roots), enforces Responsible AI (RAI) guardrails, and integrates LangFuse for real-time observability.

1.2 Business Context

A global multi-specialty hospital network manages a high volume of patient discharges across regions. Discharge documents arrive in multiple formats (PDF, DOCX, scanned handwritten reports, structured EHR exports) and multiple languages (English, Hindi,

Spanish, German). Manual review is time-consuming, error-prone, and dependent on senior clinical staff. Failures lead to re-admissions, medication errors, missed follow-ups, and regulatory non-compliance.

1.3 Project Objectives

- Ingest patient discharge reports, lab reports, and hospital bills in multiple formats and languages.
 - Extract and translate all incoming documents into English using MCP Sampling.
 - Validate discharge documents against a Mock EHR system and configurable YAML rules.
 - Use MCP Elicitation to interactively collect missing field values from human reviewers.
 - Index discharge documents into a FAISS vector store for RAG-based question answering.
 - Generate structured validation reports (JSON + HTML/PDF) with full audit trails.
 - Stream patient-friendly discharge summaries via A2A Streaming for progressive UI delivery.
 - Support HITL feedback, corrections, and re-run via a Streamlit dashboard.
 - Expose all clinical tools through two MCP servers demonstrating all six MCP primitives.
 - Orchestrate agents across three frameworks (LangGraph, ADK, Agno) via the A2A Protocol.
 - Enforce RAI guardrails (PII redaction, hallucination check, prompt injection guard).
 - Provide real-time LangFuse tracing for every agent, tool call, LLM generation, and guardrail event.
-

2. Project Requirements

2.1 Clinical Document Monitoring [Google ADK]

Design a Discharge Monitor Agent using Google ADK that scans the incoming document folder for new patient discharge files (discharge reports, lab reports, hospital bills). No live EHR integration required — file-system simulation is sufficient.

MCP Roots Integration (mandatory):

- The agent registers the input folder as a Root URI (e.g., `file:///data/input`) when opening the MCP connection.

- The Clinical Watcher MCP Tool calls `ctx.list_roots()` to discover authorized folders at runtime — no raw paths passed as tool parameters.
- Server enforces path-traversal prevention using `Path.relative_to()` checks; requests outside the declared root are rejected.

2.2 Clinical Data Extraction [LangGraph]

Design a Clinical Extractor Agent using LangGraph (StateGraph + MemorySaver) that extracts structured and unstructured clinical data from discharge reports, lab reports, medication lists, clinical notes, and hospital bills. Must support multiple languages and multi-modal content.

MCP Resources — Primary MCP Server must expose:

Resource URI	Type	Content
<code>resource://clinical-rules/completeness</code>	TextResource	Completeness rules from rules.yaml
<code>resource://clinical-rules/cross-validation</code>	TextResource	Cross-validation rules from rules.yaml
<code>resource://discharge-report/{patient_id}</code>	FileResource	Raw discharge document text
<code>resource://lab-report/{patient_id}</code>	FileResource	Raw lab report text
<code>resource://report-template/html</code>	FileResource	HTML discharge summary template
<code>resource://medical-abbreviations</code>	TextResource	Abbreviation expansion dictionary
<i>Table 1 — MCP Resources exposed by the Primary Clinical Tools Server</i>		

MCP Prompts — Primary MCP Server must expose:

Prompt Name	Parameters	Used By Agent
<code>discharge-extraction-prompt</code>	language, doc_types	Clinical Extractor Agent
<code>ehr-cross-validation-prompt</code>	patient_id	Validation Agent

abbreviation-normalization-prompt	source_language	Normalizer Agent
summary-generation-prompt	risk_level, audience	Summary Generator Agent
rag-answer-prompt	context_length	Agno RAG Generation Agent
Table 2 — MCP Prompts exposed by the Primary Clinical Tools Server		

2.3 Clinical Language Normalization [LangGraph + MCP Sampling]

Design a Clinical Normalizer Agent using LangGraph that translates extracted clinical content (Hindi, Spanish, German to English) and normalizes medical abbreviations (BID = twice daily, PO = by mouth, etc.). Must include translation confidence score in output.

MCP Sampling Integration (mandatory):

- The Medical Lang Bridge MCP Tool issues a `ctx.session.create_message()` sampling request to the calling agent’s LLM client, including the text to translate and ModelPreferences with model hints (nova-lite for multilingual, command-r-plus for English).
- The calling LangGraph agent implements a `sampling_callback` that reads the server’s model hints, routes to the appropriate LiteLLM model, performs inference, and returns the translated text as `CreateMessageResult`.
- This separates LLM resource management (client responsibility) from tool logic (server), demonstrating the Sampling primitive’s architectural intent.

2.4 Clinical & Business Validation [LangGraph]

2.4.1 Clinical Completeness Validation

Validate discharge reports, lab reports, and bills against `configs/rules.yaml` (fetched as MCP Resource at runtime). If any blocking field is missing, auto-generation of the discharge summary is blocked and HITL must intervene.

Document	Required Fields	Blocking If Missing
Discharge Report	patient_id, patient_name, age, gender, address, admission_date, discharge_date, ward, bed_no, doctors,	patient_id, patient_name, discharge_diagnosis,

	discharge_diagnosis, medications, adr_allergy_info, follow_up_appointments, discharge_instructions, discharge_approved_by, discharge_approved	discharge_approved, medications
Lab Report	patient_id, vendor_name, lab_name, report_date, tests	patient_id, tests
Bill	patient_id, hospital_name, billing_date, line_items, total_amount, payment_status	patient_id, total_amount, payment_status
Prescription (per med)	sl_no, medicine_name, strength, dosage, frequency, route, period, remarks, total_quantity	medicine_name, strength, frequency, route
<i>Table 3 — Completeness validation fields by document type</i>		

MCP Elicitation Integration (mandatory):

- When the Rules Engine Tool detects non-blocking missing fields, it calls `ctx.elicit()` with a Pydantic-based schema describing the missing fields and their types.
- Tool handles all three elicitation outcomes: accept (use reviewer input and continue), decline (mark unresolved, flag for HITL), cancel (abort and escalate).
- The HITL Streamlit dashboard implements the `elicitation_callback` — renders a dynamic form to the human reviewer and returns `ElicitResult`.

2.4.2 Cross-Validation vs. Mock EHR / Care Plan / Labs

Rule ID	Severity	Check Description	Action
med_omission_check	Warning	Discharge meds differ from EHR medication history	Flag for review

allergy_contradiction_check	Critical	Prescribed med conflicts with known allergy	Block discharge
diagnosis_mismatch_check	Warning	Discharge diagnosis differs from EHR care plan	Flag for review
follow_up_missing_check	Critical	Follow-up not documented despite care plan requirement	Block discharge
lab_follow_up_mismatch_check	Warning	Abnormal lab values have no documented action	Flag for review
discharge_approval_check	Critical	Discharge not approved by treating physician	Block discharge
bill_settlement_check	Critical	Bill not PAID or lacking insurance guarantee letter	Block discharge
<i>Table 4 — Cross-validation rules (loaded from configs/rules.yaml via MCP Resource)</i>			

2.5 Audit & Risk Report Generation

Design an agent that generates a clinician/admin-friendly discharge audit report. Output: JSON (system consumption) + HTML/PDF (clinician-friendly). Include: missing fields, EHR discrepancies, medication conflicts, translation confidence, risk level (Low/Medium/High), recommendation (Approve/Edit/Reject), audit trail with LangFuse trace IDs, bill amount and payment status.

2.6 Agentic RAG — Clinical Question Answering [Agno]

Provide a context-aware QA assistant for hospital administrators. For out-of-context questions the response must be: "I don't know — this information is not available in the

patient records.” The RAG Generation Agent must fetch its answer prompt via MCP Prompts (`get_prompt("rag-answer-prompt")`) rather than using a hardcoded string.

RAG Agent Role	Framework	Description
Indexing Agent	Agno	Parses and indexes discharge documents into FAISS vector store
Retrieval Agent	Agno	Converts questions to embeddings; retrieves top-k chunks
Augmentation Agent	Agno	Re-ranks retrieved chunks by keyword relevance
Generation Agent	Agno	Generates grounded responses; prompt fetched via MCP Prompts
Reflection Agent	Agno	Scores quality via RAG Triad: Faithfulness / Answer Relevance / Context Relevance
Table 5 — Five Agentic RAG roles, all implemented using Agno framework		

Agno-specific requirements: use `agno.Agent` with `MultiMCPTools` , SQLite-backed session persistence (`SqliteDb` , last 3 turns as context), async `arun()` invocation, exposed as A2A agent on port 8105.

3. Agent Roles & Framework Assignment

Agent	Framework	A2A Port	MCP Primitives Used	A2A Mode
Discharge Monitor Agent	Google ADK	8103	Tools + Roots	Non-streaming
Clinical Extractor Agent	LangGraph	8100	Tools + Resources + Prompts	Non-streaming
Clinical Normalizer Agent	LangGraph	8102	Tools + Sampling + Prompts	Non-streaming

Clinical Validation Agent	LangGraph	8101	Tools + Elicitation + Resources	Non-streaming
Discharge Summary Generator	Google ADK	8104	Tools + Prompts	STREAMING
Clinical RAG Q&A Agent	Agno	8105	MultiMCPTools + Prompts	STREAMING
Host Orchestrator	Google ADK	8083	A2A Client (streaming-capable)	A2A Client
<i>Table 6 — All agents with framework assignment, A2A port, and MCP primitives used</i>				

3.1 Agent Tools

Tool Name	MCP Primitive	Purpose
Clinical Watcher Tool	Tool + Roots	Detects new discharge files within Roots-scoped workspace
Clinical Data Harvester Tool	Tool	Extracts text, tables, images from clinical documents
Medical Lang Bridge Tool	Tool + Sampling	Translates and normalizes clinical language via LLM sampling
Clinical Rules Engine Tool	Tool + Elicitation	Validates completeness; elicits missing data from reviewer
EHR Validation Tool	Tool	Cross-checks discharge data against Mock EHR REST API
Clinical Insight Reporter Tool	Tool + Resources	Generates JSON and HTML audit/risk reports

Risk Score Tool (Secondary)	Tool	Computes composite discharge risk score — Secondary MCP Server :8201
Population Benchmarks Tool (Secondary)	Tool	Returns readmission rates and benchmarks — Secondary MCP Server :8201
<i>Table 7 — All MCP tools (last two on Secondary Analytics MCP Server, port 8201)</i>		

4. Multi-Server MCP Architecture

Agents must connect to two MCP servers simultaneously, demonstrating multi-server MCP connectivity as taught in the course:

MCP Server	Port	Transport Path	Primitives	Key Tools
Primary: Clinical Tools Server	8200	/clinicaltools	Tools, Resources, Prompts, Sampling, Elicitation, Roots	Watcher, Harvester, Lang Bridge, Rules Engine, EHR Validator, Reporter
Secondary: Analytics Server	8201	/analyticstools	Tools	calculate_risk_score, get_population_benchmarks, generate_risk_heatmap
<i>Table 8 — Dual MCP server configuration</i>				

MCP Primitive Coverage — All 6 primitives must be demonstrable:

Primitive	Where Implemented	Key APIs Used
-----------	-------------------	---------------

Tools	Primary MCP Server (8200)	<code>mcp.tool()</code> decorator, tool invocation
Resources	Primary MCP Server (8200)	<code>mcp.resource()</code> , <code>list_resources()</code> , <code>read_resource()</code>
Prompts	Primary MCP Server (8200)	<code>mcp.prompt()</code> , <code>list_prompts()</code> , <code>get_prompt()</code>
Sampling	Medical Lang Bridge Tool	<code>ctx.session.create_message()</code> , <code>sampling_callback</code> , <code>ModelPreferences</code>
Elicitation	Clinical Rules Engine Tool + HITL Dashboard	<code>ctx.elicit()</code> , <code>ElicitResult</code> , <code>elicitation_callback</code> , <code>accept/decline/cancel</code>
Roots	Discharge Monitor Agent <-> Watcher Tool	<code>ctx.list_roots()</code> , <code>Root(uri=...)</code> , path-traversal prevention
<i>Table 9 — All 6 MCP primitives and their implementation location</i>		

5. Agent-to-Agent (A2A) Protocol

All agents must expose an AgentCard at `GET /.well-known/agent.json` and implement the A2A Protocol for discovery, invocation, and message exchange. Authentication via shared secret header `X-Agent-Auth-Token` is required on all A2A servers.

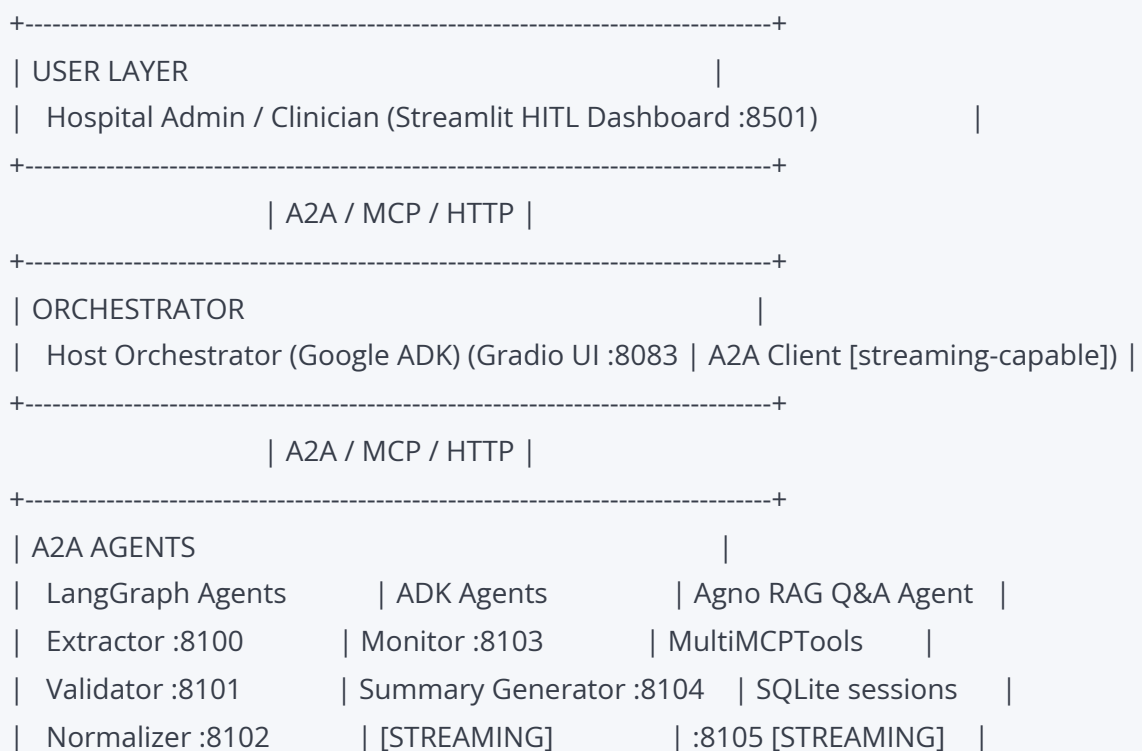
Streaming vs. Non-Streaming: At least two agents must support A2A Streaming:

Agent	Streaming Mode	Client Must Use	Progressive Output
Discharge Summary Generator (8104)	<code>streaming=True</code>	<code>send_message_streaming()</code> async for event in stream	Section-by-section summary: patient -> meds

			-> labs -> bill -> instructions
Clinical RAG Q&A Agent (8105)	streaming=True	send_message_streaming() async for event in stream	Token-by-token answer streaming in HITL Q&A page
All Other Agents	streaming=False	send_message() await response	Single final artifact returned
Table 10 — A2A streaming vs. non-streaming agent configuration			

6. System Architecture Diagram

The diagram below shows all services, their connections, and the protocols used between them. Colour coding: Blue/Navy = LangGraph, Orange = Google ADK, Purple = Agno, Green = MCP servers, Light grey = Data/storage layer.



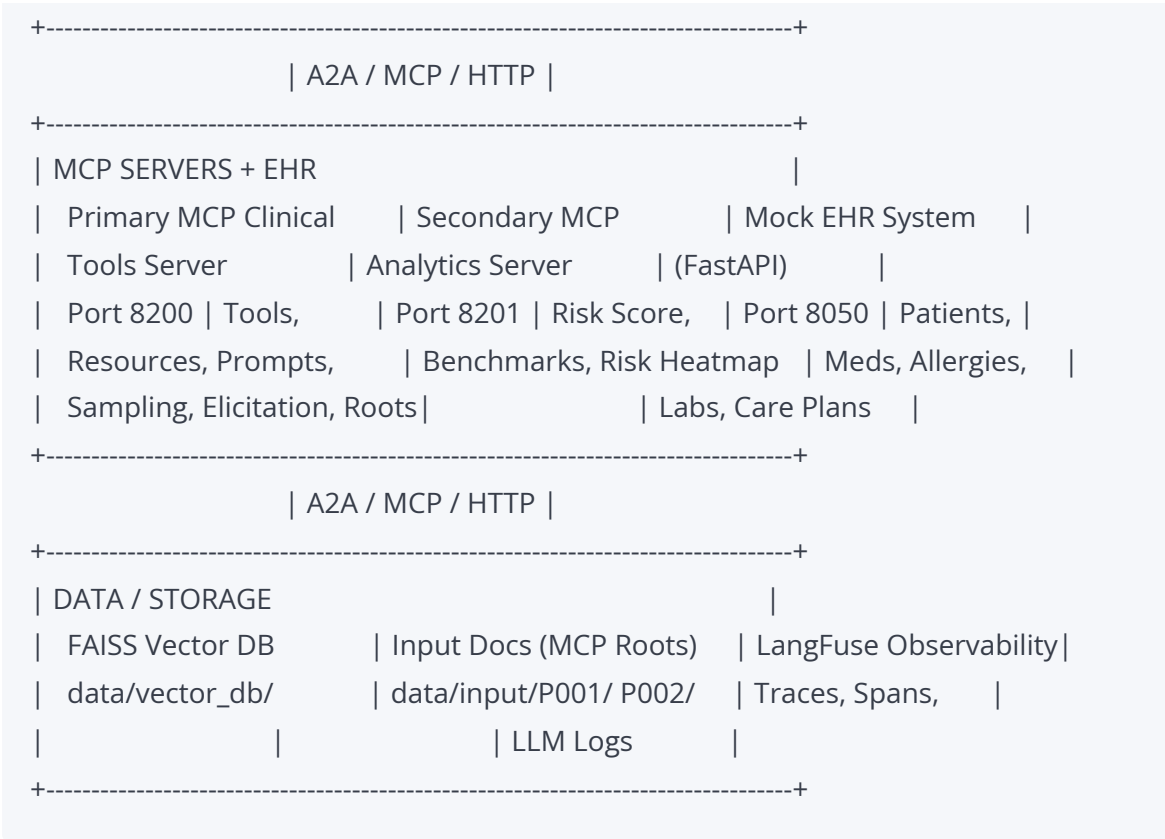


Figure 1 — System Architecture: Users -> Host Orchestrator -> A2A Agents -> Dual MCP Servers + Mock EHR

Colour	Framework / Layer
Navy / Blue	LangGraph Agents
Orange	Google ADK Agents + Host
Purple	Agno RAG Q&A Agent
Green	MCP Servers (Primary + Secondary)
Light Grey	Data / Storage Layer
Table 11 — Colour legend for architecture diagram	

7. Responsible AI Guardrails & LangFuse Observability

7.1 RAI Guardrails

Guardrail	Module	Trigger Condition	Action
-----------	--------	-------------------	--------

PII/PHI Redaction	PIIRedactor	Any text containing patient name, phone, Aadhaar, PAN	Mask before logging/API calls
Hallucination Check	HallucinationChecker	RAG-generated response (faithfulness < 0.7)	Block response; request regeneration
Prompt Injection Guard	PromptInjectionGuard	User query matches injection patterns	Sanitize or reject; log alert
Toxicity Filter	ToxicityFilter	LLM output in clinical instructions	Filter before including in summary
HITL Escalation	GuardrailManager	risk_level=High or discharge_blocked=True	Mandatory human review; no auto-approve
<i>Table 12 — Responsible AI guardrails with trigger conditions and actions</i>			

7.2 LangFuse Observability Requirements

- End-to-end trace ID per discharge case, passed through all agents via message metadata.
- Per-agent spans with latency, input payload, and output payload.
- Per-tool-call spans for every MCP tool invocation (tool name, parameters, result, duration).
- LLM generation events: model name, prompt, response, token counts, estimated cost.
- Sampling events: server model preferences, client model selected, translation result.
- Elicitation events: schema sent, reviewer response, action taken (accept/decline/cancel).

- Guardrail intervention spans: guardrail name, check result, content blocked/allowed.
- Error spans: exception type, stack trace, fallback action taken.

8. HITL Dashboard — Streamlit (5 Pages)

Page	Title	Key Features
1	Document Viewer	Patient selector · Tab view (Discharge/Lab/Bill) · Language detection badge · Structured data preview · Process trigger button
2	Validation Report	Completeness score (colour-coded) · Cross-validation issues table · Risk level badge · Recommendation · Discharge blocked indicator · LangFuse trace link
3	HITL Corrections	Editable medication table (<code>st.data_editor</code>) · Elicitation Response Form (dynamic, schema-driven) · Risk label override · Approval decision · Save feedback · Re-run Validation (streaming)
4	RAG Q&A	Patient filter · Example query buttons · Prompt injection indicator · Streaming response display · Source docs panel · RAG Triad quality metrics
5	Discharge Summary	Patient-friendly summary for auto-approved cases · Plain-English prescription table · Colour-coded lab results · Export JSON / HTML / PDF · LangFuse trace link
Table 13 — HITL Dashboard pages and their key features		

9. Technology Stack

Category	Details
----------	---------

Backend & Frameworks	Python 3.11+ · LangGraph · Google ADK · Agno
Vector Database	FAISS (primary) · Qdrant / Weaviate (optional)
OCR / Document Parsing	Tesseract (optional) · text extraction from .txt / .pdf / .docx
LLM Models	AWS Bedrock Nova Lite (primary) · Cohere Command R+ (fallback) · sentence-transformers/all-MiniLM-L6-v2 (embeddings)
LLM Gateway	LiteLLM — unified interface to AWS Bedrock + Cohere APIs
MCP Protocol	Primary MCP Server :8200 (6 primitives) · Secondary Analytics Server :8201 · mcp-use for multi-server client
A2A Protocol	a2a-sdk - Non-streaming + Streaming - Push Notifications - Shared-secret auth
Mock EHR System	FastAPI :8050 · 5 JSON data files (patients, medications, allergies, labs, care_plans)
Frontend / UI	Streamlit (HITL Dashboard, 5 pages, :8501) · Gradio (Host Orchestrator UI, :8083)
Observability	LangFuse — end-to-end traces - per-agent spans - LLM generation logs - guardrail spans
RAI Guardrails	PIIRedactor · HallucinationChecker (LLM-as-a-Judge) · PromptInjectionGuard · ToxicityFilter
Configuration	configs/rules.yaml · configs/prompts.yaml · configs/agent_config.yaml
Deployment	NuvePro Lab
<i>Table 14 — Complete technology stack</i>	

10. System Port Map

Service	Port	Protocol	Framework	Role
---------	------	----------	-----------	------

Mock EHR System	8050	HTTP/REST	FastAPI	Patient data, meds, labs, care plans
Primary MCP Clinical Tools Server	8200	MCP streamable-HTTP	FastMCP	6 tools + Resources + Prompts + Sampling + Elicitation + Roots
Secondary MCP Analytics Server	8201	MCP streamable-HTTP	FastMCP	Risk score, benchmarks, heatmap tools
Clinical Extractor Agent (A2A)	8100	A2A non-streaming	LangGraph	Extract structured data from docs
Clinical Validation Agent (A2A)	8101	A2A non-streaming	LangGraph	Completeness + EHR cross-validation
Clinical Normalizer Agent (A2A)	8102	A2A non-streaming	LangGraph	Translate + normalize medical terms
Discharge Monitor Agent (A2A)	8103	A2A non-streaming	Google ADK	Monitor input folder via MCP Roots
Summary Generator Agent (A2A)	8104	A2A STREAMING	Google ADK	Generate patient-friendly summary
Clinical RAG Q&A Agent (A2A)	8105	A2A STREAMING	Agno	5-role RAG with MultiMCPTools + SQLite
Host Orchestrator	8083	Gradio + A2A client	Google ADK	Coordinate all agents
Streamlit HITL Dashboard	8501	HTTP	Streamlit	5-page human review interface
<i>Table 15 — Complete system port map</i>				